CHAPTER 1
ABSTRACT

Vehicular ad hoc networks (VANETs) let cars exchange safety and traffic messages directly with each other, without a base station. Whether these messages arrive depends on how many vehicles are on the road, how fast they drive, how much data they send and how far their radios reach. This project measures these four effects with the NS-3 network simulator (ns-3.48). We built a 2 km, four-lane highway on which vehicles drive in both directions, route with AODV over an IEEE 802.11p radio and exchange ten constant-rate UDP flows for 100 seconds. Starting from a baseline of 60 vehicles, 20 m/s, 100 kbit/s per flow and a 150 m range, we varied one factor at a time and ran every test point with five random seeds (95 runs), measuring packet delivery ratio (PDR), end-to-end delay, throughput and packet loss with FlowMonitor. Communication range had the strongest effect (PDR 5.8% at 50 m to 53.1% at 250 m), followed by speed (46.1% at 10 m/s to 32.3% at 25 m/s) and load (39.2% at 50 kbit/s to 19.2% at 750 kbit/s). Density helped only up to 60–80 vehicles. A second variation with a different baseline and eight seeds (200 runs) confirmed the trends. Finally we simulated the combination of the best value of every factor (80 vehicles, 10 m/s, 50 kbit/s, 250 m). Over ten seeds it raised PDR from 36.0% to 59.1% and cut the delay from 69.2 ms to 20.7 ms, but it stayed well below the 71% that a simple multiplication of the single-factor gains predicts, because the factors interact.

CHAPTER 2
OBJECTIVES

The aims of this assessment were:

- To learn how socket programming works in NS-3: sending and receiving over UDP, binding a socket to a port and handling each arriving packet in a receive callback.
- To build a highway VANET in NS-3 with traffic in both directions, using AODV routing over an IEEE 802.11p radio and UDP for the data.
- To measure packet delivery ratio, end-to-end delay, throughput and packet loss rate on the data flows only, leaving out the AODV control traffic.
- To study the effect of vehicle density, vehicle speed, traffic load and communication range one at a time, with several random seeds and error bars.
- To check that the results are correct and repeatable: fix bugs found on the way, repeat runs and confirm the trends with a second, independent parameter variation.
- To see inside individual runs, using packet-level traces, animations and NetAnim to show vehicle positions, data hops, route discovery and route errors.
- To find the best tested value of each factor, run those values together in the simulator and work out how much each factor adds to the combined result.

CHAPTER 3
INTRODUCTION

3.1 Network Programming and the Application

Network programming is about writing programs that run on separate devices and exchange messages. In our project those devices are vehicles. A car can warn the cars behind it about sudden braking, an accident or a traffic jam, and it can share its speed and position to help traffic flow. The first kind of use is **road traffic safety**, where warnings have to arrive quickly and reliably. The second is **traffic efficiency**, where the information helps vehicles choose a speed and route [2], [3].

In a vehicular ad hoc network (VANET), each vehicle has a small radio and also forwards traffic for other vehicles, so a message can travel from car to car until it reaches its destination. We studied this in a simulator because a run can be repeated exactly with the same seed and every packet can be followed. A program defines the road, the vehicles and their radios. We follow the evaluation framework of Malnar and Jevtić [1], who use NS-3 to measure how VANET performance changes with the conditions.

3.2 Sockets, Protocols and System Calls

A socket is the standard interface a program uses to send and receive network data. On Linux, a program creates a socket and uses system calls such as *socket()*, *bind()*, *sendto()* and *recvfrom()* [22], [23]. NS-3 has a Socket class that works the same way [24]. In our scenario every sender runs an NS-3 OnOffApplication. It creates a UDP socket, connects it to the receiver's address and port and calls *Send()* at a constant rate. Each receiver runs a PacketSink, which binds a UDP socket to its port and registers a callback that is called for every arriving packet.

TCP retransmits lost data and lowers its sending rate when the network is congested. Both add delay, and safety applications need delays of the order of 100 ms or less [3], [4]. UDP sends each packet once and does not wait for an acknowledgement, so we use it for all our data flows. This is a choice made for our simulation: real DSRC safety messages normally use the WAVE Short Message Protocol of IEEE 1609.3 instead of UDP/IP [4]. AODV is a network program too. It sends its route requests, replies and errors as UDP packets on port 654, over the same radio channel as the data.

3.3 Vehicular Ad Hoc Networks and IEEE 802.11p

A VANET has no fixed access point. Vehicles move fast, often in opposite directions, so the links between them last only a few seconds and the topology changes all the time [2], [9]. On a highway the network is almost one-dimensional: a message must be passed along the road, and a gap between groups of vehicles that is longer than the radio range splits the network into parts that cannot reach each other [12].

The radio standard for vehicles is IEEE 802.11p, which became part of IEEE 802.11 as the "outside the context of a BSS" (OCB) mode [5], [6]. It works in the 5.9 GHz band with 10 MHz channels, which makes the symbols longer and more robust to the fast-changing radio channel of moving vehicles. In the United States it is the basis of Dedicated Short-Range Communications (DSRC) [4]. Recent ns-3 versions no longer include the *wave* module that provided the OCB MAC, so we configure the 802.11p physical layer through the *wifi* module and use an ad hoc MAC.

3.4 Routing and Quality of Service

We use AODV (Ad hoc On-Demand Distance Vector) routing [7], [8]. AODV looks for a route only when a vehicle has data to send: it floods a route request (RREQ), the destination answers with a route reply (RREP) along the reverse path, and when a link on the route breaks a route error (RERR) is sent so that the source can search again. Hello messages tell each vehicle which neighbours are in range. Route discovery costs time and channel capacity, and in a VANET it must be repeated often because links break quickly.

Quality of Service (QoS) describes how well the network serves the traffic. We use four measures: **packet delivery ratio** (the share of packets that arrive), **end-to-end delay** (time a packet takes), **throughput** (data delivered per second) and **packet loss rate** (the share that never arrives). Safety applications in particular need a high delivery ratio and a delay of the order of 100 ms or less [3], [4].

3.5 Tools and Languages Used

- NS-3 (version 3.48), a discrete-event network simulator written in C++ [20], built on macOS. The scenario was first written for ns-3.45 and runs unchanged on ns-3.48.
- C++ for the simulation program *vanet-highway.cc*, using the ns-3 wifi, aodv, internet, mobility, applications, flow-monitor and netanim modules [24].

- FlowMonitor, an NS-3 tool that records packets sent, packets received and delay for every flow [21].
- Bash for the experiment runners (*run_experiments.sh*, *make_animations.sh*), which run the simulations in parallel with xargs.
- Python 3 with numpy and matplotlib for merging the results, computing mean and standard deviation and drawing graphs (*plot_results.py*) and for animating traced runs (*animate_trace.py*, *animate_sweep.py*).
- NetAnim 3.110, the ns-3 animator, for viewing individual runs, and git for version control.

CHAPTER 4
LITERATURE REVIEW

We studied twenty-one refereed papers, standards and book chapters (IEEE, ACM, Springer and other refereed venues) on vehicular networking, routing, mobility, medium access and simulation. They are grouped by theme below and summarised in Table 1.

Evaluation framework. Malnar and Jevtić propose a framework for evaluating VANETs in NS-3: a road scenario, a routing protocol and a set of metrics (delivery ratio, delay, throughput) measured while one condition at a time is changed [1]. Our study follows this approach.

Vehicular networks and standards. Hartenstein and Laberteaux give a tutorial survey of VANETs and their safety and efficiency applications [2]. Karagiannis et al. review requirements, architectures and standards, including the delay needs of safety applications [3]. Kenney describes the DSRC standards in the United States [4], and Jiang and Delgrossi explain how IEEE 802.11p adapts Wi-Fi to vehicles [5]; the amendment itself is [6].

Routing. Perkins and Royer introduced AODV [7], later standardised as RFC 3561 [8]. Li and Wang survey routing in VANETs and show that fast topology change is the main difficulty [9]. Broch et al. found that mobility and load strongly affect ad hoc routing [10]. Ni et al. describe the broadcast storm problem: flooding in a dense network causes redundant transmissions and collisions [11], which is what happens to RREQ floods. Wisitpongphan et al. show that sparse highway networks are often disconnected and need store-carry-forward ideas [12].

Mobility. Härri et al. classify mobility models for VANETs [13], and Camp et al. survey mobility models for ad hoc networks [25]. Bai et al. show that the mobility pattern, and especially the relative speed of nodes, changes routing performance [14].

Medium access and capacity. Bianchi's model shows that Wi-Fi throughput is limited by stations competing and colliding on the channel [15]. Gupta and Kumar show that the capacity per node of a multi-hop wireless network falls as the network grows [16]. Torrent-Moreno et al. measure how reception in 802.11 vehicular networks falls with distance and load [17].

Simulation tools. Veins couples a network simulator with the SUMO road traffic simulator [18], [19]. Riley and Henderson describe ns-3 [20], and Carneiro et al. describe FlowMonitor [21].

Table 1. Summary of the reviewed literature

Ref.	Work (year)	Main contribution	Relevance to this work
[1]	Malnar & Jevtić (2020)	NS-3 framework for VANET performance evaluation	Framework we follow
[2]	Hartenstein & Laberteaux (2008)	Tutorial survey of VANETs	Safety and efficiency uses
[3]	Karagiannis et al. (2011)	Requirements, architectures, standards	Delay needs of safety apps
[4]	Kenney (2011)	DSRC standards in the USA	Radio band and channel
[5]	Jiang & Delgrossi (2008)	IEEE 802.11p for vehicles	PHY we configure
[6]	IEEE 802.11p (2010)	WAVE amendment to IEEE 802.11	Standard used
[7]	Perkins & Royer (1999)	On-demand route discovery (AODV)	Routing protocol used
[8]	Perkins et al. (2003)	RFC 3561: AODV specification	Reference for AODV
[9]	Li & Wang (2007)	Survey of VANET routing	Why links break
[10]	Broch et al. (1998)	Comparison of ad hoc routing protocols	Mobility and load matter
[11]	Ni et al. (1999)	Broadcast storm problem	Cost of RREQ floods
[12]	Wisitpongphan et al. (2007)	Routing in sparse VANETs	Disconnection at low density
[13]	Härri et al. (2009)	VANET mobility models	Choice of mobility model
[14]	Bai et al. (2003)	Impact of mobility on routing	Effect of speed
[15]	Bianchi (2000)	Model of 802.11 DCF throughput	Channel saturation
[16]	Gupta & Kumar (2000)	Capacity of wireless networks	Multi-hop capacity limit
[17]	Torrent-Moreno et al. (2004)	Reception rates in 802.11 VANETs	Effect of load and distance
[18]	Sommer et al. (2011)	Veins: coupled network and road simulation	Alternative tool chain
[19]	Krajzewicz et al. (2012)	SUMO road traffic simulator	Realistic traffic (future work)
[20]	Riley & Henderson (2010)	The ns-3 simulator	Simulation platform
[21]	Carneiro et al. (2009)	FlowMonitor for ns-3	Measurement tool used

In short, the effects of mobility, density and load on ad hoc routing are well known in general, and NS-3 offers a framework to measure them. However, [1] and [10] change one condition at a time, which does not show whether the best values work together, and the *wave* module that provided 802.11p in older ns-3 releases is no longer in current ns-3.

CHAPTER 5
RESEARCH GAP

Chapter 4 points to four gaps:

1. **Single runs.** One run per setting is not enough for a mobile network, because the result depends strongly on where the vehicles start. In our density sweep, seed 1 gives 40.3% PDR at 100 vehicles, while the mean over five seeds is 29.1% (Section 9.3).
2. **Factors studied alone.** Density, speed, load and range are usually varied one at a time [1], [10]. That leaves open whether the best value of each factor still helps when the others are also at their best. Our search for the best values also went one factor at a time, but we then ran the combined values in the simulator and measured the result.
3. **Current tools.** The ns-3 wave module that provided the 802.11p MAC has been removed, so an 802.11p scenario on current ns-3 has to be set up through the wifi module instead.
4. **Separating range from fading.** With a realistic fading model, range and loss are mixed together. A hard range cut-off isolates the effect of how far the radio reaches.

Selected research gap. We choose one gap, combining the first, second and third: there is a lack of a reproducible, multi-seed study of AODV over IEEE 802.11p in current ns-3 that measures the effect of density, speed, load and range on delivery ratio, delay, throughput and loss, and that measures (instead of estimating) the combined best configuration and the contribution of each factor to it. The fourth point is a design choice and is treated as a limitation.

CHAPTER 6
PROBLEM STATEMENT

Problem statement. To design, build and run in NS-3 a highway VANET in which vehicles route with AODV over IEEE 802.11p and exchange UDP traffic; to measure how packet delivery ratio, end-to-end delay, throughput and packet loss change with vehicle density, vehicle speed, traffic load and communication range; and to find and verify the configuration that gives the most reliable delivery.

In simple terms. Let N be the number of vehicles, v their mean speed, R the sending rate of each flow and r the radio range. For each factor we change its value while the other three stay fixed and record the PDR, P(N, v, R, r). We then take the tested value of each factor with the highest mean PDR, combine those four values and run the combination in the simulator. The search goes one factor at a time over a few tested values, so it may not find the best possible combination.

Research questions. (1) What happens to PDR, delay, throughput and loss as each factor changes? (2) Which factor has the largest effect over the values we tested? (3) Do the trends hold with a different baseline and more seeds? (4) What PDR and delay does the combination of the best values give, and is it what the single-factor results predict? (5) Does each factor still help once the others are at their best?

Scope and assumptions. This is a simulation of a straight two-way highway with constant-velocity vehicles. The radio reaches exactly r metres (no fading or obstacles). There are no roadside units; all traffic is vehicle-to-vehicle. Traffic is unicast, constant-rate UDP between random vehicle pairs, not broadcast safety beacons. AODV and the ns-3 wifi model are used with their default settings, and no ns-3 library file is modified.

CHAPTER 7
METHODOLOGY / IMPLEMENTATION

7.1 Proposed Architecture

The scenario is a straight highway 2000 m long with two lanes in each direction, 5 m apart (Fig. 1). Vehicles are placed at random positions along the road; vehicle i uses lane i mod 4, so half drive east and half drive west. Each vehicle drives at the mean speed multiplied by a random factor between 0.8 and 1.2, so vehicles overtake each other. A vehicle that drives off one end of the road re-enters at the other end, so the number of vehicles on the road stays constant during the run. Ten vehicles are chosen as senders; each sends a constant-rate UDP flow to a randomly chosen receiver. All other vehicles relay packets.

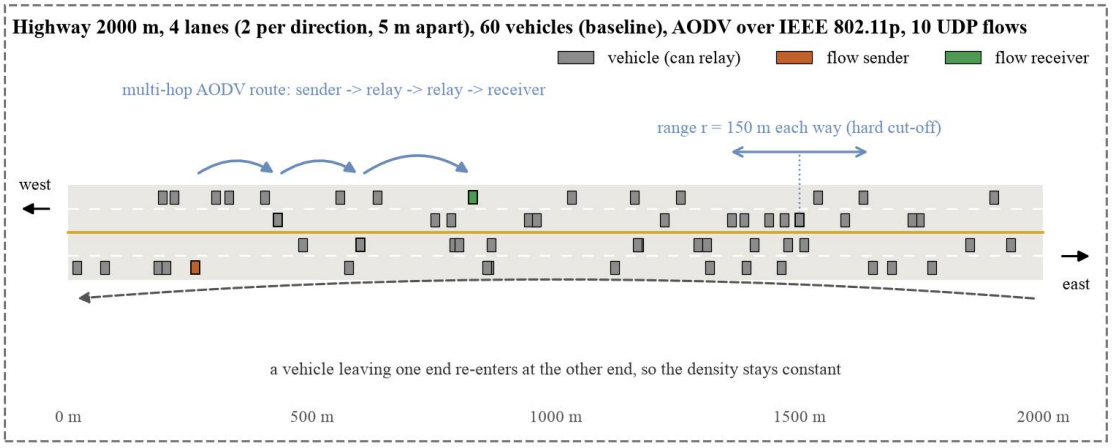


Fig. 1. Simulation scenario: two-way highway and multi-hop AODV routes

The work was done in stages. Each stage is saved in the git history of the project (Table 2).

Table 2. Development stages and the programs built

Stage	Program / output	Change	Role
1	vanet-highway.cc, run_experiments.sh, plot_results.py	Highway scenario, sweep runner, graphs	Baseline study
2	run_experiments.sh, plot_results.py	Sweep values and baseline configurable; separate plots folder	Enables new variations
3	vanet-highway.cc (--trace), animate_trace.py	Packet-level event trace and single-run animation	Visual check
4	vanet-highway.cc, animate_sweep.py, make_animations.sh	Wrap-around bug fixed; NetAnim output; sweep animations; all results regenerated	Final main results
5	results/plots2	Second variation: new baseline, wider ranges, 8 seeds	Robustness check
6	results/optimal	Combined best configuration and leave-one-out runs, 10 seeds	Optimal solution

Node architecture

Every vehicle has the same protocol stack (Fig. 2). Senders and receivers differ only in the application they run. The data passes down the sender's stack, is relayed by other vehicles at the network layer by AODV, and passes up the receiver's stack. FlowMonitor watches the IP layer of every vehicle and records each packet of every flow.

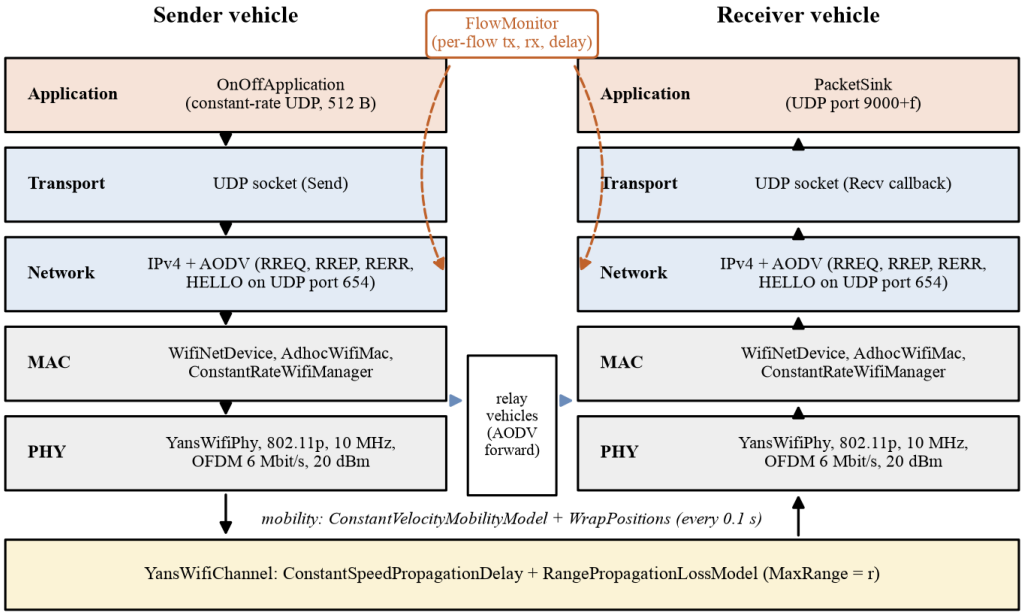


Fig. 2. Protocol stack of a vehicle and the path of a data packet

Table 3. Roles of the vehicles

Role	Responsibility
Every vehicle (N)	Moves at constant velocity; runs IPv4, AODV and an 802.11p radio; can relay packets
Sender (10 distinct vehicles)	OnOffApplication: constant-rate UDP, 512-byte payload, to port 9000 + f; starts at a random time in 10–11 s, stops at 95 s
Receiver (random, not the sender)	PacketSink bound to UDP port 9000 + f; receive callback counts packets
Relay (any vehicle on the route)	Forwards data and AODV messages hop by hop
FlowMonitor (all vehicles)	Records packets sent and received and their delay for each flow
Trace / NetAnim (optional)	Records positions, frames on air and AODV messages for animations

7.2 Mathematical Model

Mobility. Vehicle i starts at a random position $x_i(0)$ on a road of length L = 2000 m, with direction $d_i = +1$ (east) or -1 (west) and speed $v_i = \bar{v} \cdot U(0.8, 1.2)$. Its position at time t is

$$x_i(t) = (x_i(0) + d_i \cdot v_i \cdot t) \mod L \qquad (1)$$

Links. With the hard range cut-off, vehicles i and j can hear each other at time t only if

$$| p_i(t) - p_j(t) | \leq r \tag{2}$$

where p is the position (x, lane) and r the communication range. With N vehicles spread over L, a vehicle has on average about

$$\bar{n} \approx 2 \cdot r \cdot N / L \tag{3}$$

neighbours. For the baseline (N = 60, r = 150 m) this is about 9 neighbours; at r = 50 m it is 3 and at r = 250 m it is 15. With only a few neighbours, a gap longer than r appears easily and splits the road into isolated groups [12].

Hops. A sender and receiver chosen at random on a road of length L are on average L/3 ≈ 667 m apart, so a route needs about

$$\bar{h} \approx L / (3 \cdot r) \tag{4}$$

hops: about 4.4 at r = 150 m, 2.7 at r = 250 m and 13 at r = 50 m. Every extra hop is another chance for a collision or a broken link.

Link lifetime. Two vehicles with relative speed Δv stay in range for at most

$$T_{link} \approx 2 \cdot r / \Delta v \tag{5}$$

For vehicles in opposite directions $\Delta v \approx 2\bar{v}$: at $\bar{v}$ = 20 m/s and r = 150 m a link lasts at most 7.5 s, at 10 m/s about 15 s. Faster vehicles therefore break routes more often and force AODV to search again [9], [14].

Offered load. Each flow sends R kbit/s in packets of S = 512 bytes, i.e. R · 1000 / (8 · S) packets per second (24.4 packets/s at 100 kbit/s). Because every packet is sent once per hop, the channel time that the flows need grows with the number of hops:

$$C \approx n_f \cdot R \cdot \bar{h} \tag{6}$$

For the baseline (n_f = 10 flows) this is about 10 × 100 × 4.4 ≈ 4.4 Mbit/s of transmissions on a 6 Mbit/s channel, before MAC overhead, AODV messages and collisions. At 750 kbit/s per flow it is many times the channel capacity, so the channel saturates [15], [16].

Metrics. Let tx_f and rx_f be the packets sent and received by data flow f, D_f the sum of the delays of its received packets and T_f the time the flow was sending. Over the n_f data flows of a run:

$$PDR = 100 \cdot \Sigma\, rx_f / \Sigma\, tx_f \tag{7}$$

$$Delay = \Sigma\, D_f / \Sigma\, rx_f \tag{8}$$

$$\text{Throughput} = (1 / n_f) \cdot \Sigma \left(rx_f \cdot 8 \cdot S / T_f \right) \quad (9)$$

$$\text{Loss} = 100 \cdot (\Sigma tx_f - \Sigma rx_f) / \Sigma tx_f \; = \; 100 - \text{PDR} \quad (10)$$

Throughput counts only the payload, so it can be compared directly with the offered rate R. Each test point is run with k seeds; we report the mean and the sample standard deviation

$$m = (1/k) \cdot \Sigma y_s, \quad \sigma = \sqrt{(\Sigma (y_s - m)^2 / (k - 1))} \quad (11)$$

Optimal configuration. For each factor j with tested values V_j, the best value is the one with the highest mean PDR, and the optimal configuration combines them:

$$x_j^* = \text{argmax}_{x \in V_j} \text{PDR}_j(x), \quad c^* = (N^*, v^*, R^*, r^*) \quad (12)$$

If the factors were independent, the PDR of c* could be estimated from the baseline PDR_0 and the gain of each factor:

$$PDR_{est} = PDR_0 \cdot \Pi_j \left(PDR_j(x_j^*) / PDR_0 \right) \quad (13)$$

Section 9.4 compares this estimate (71.0%) with the value measured in the simulator.

7.3 Network Communication Model

All data flows are UDP flows between vehicles that are usually out of direct radio range, so packets are passed along hop by hop by other vehicles on routes found by AODV. Each sender is an NS-3 OnOffApplication set to be always on, so it sends a steady stream at its DataRate with a fixed PacketSize. Senders start at a random time between 10 and 11 s, so AODV hello messages have already filled the neighbour tables, and stop at 95 s, leaving 5 s for packets still on the way. Receivers run a PacketSink with a UDP socket bound to the flow's port.

The flow of one packet is: the OnOff application calls Send() on its UDP socket; IPv4 asks AODV for a route (and buffers the packet while AODV searches); the packet is handed to the 802.11p MAC, sent at 6 Mbit/s and acknowledged by the next hop; relays repeat this until the receiver's socket gets the packet and its callback runs. The messages used are listed in Table 4. AODV messages use the same channel as the data, so they compete with it. FlowMonitor recognises the data flows by their destination port and leaves the AODV flows out of the results.

Table 4. Messages and ports used in the network

Message	From → to	Port	When	Purpose
UDP data (512 B payload)	Sender → receiver (multi-hop)	9000–9009	Constant rate, 10–95 s	Application traffic
RREQ	Source → all (flooded)	654	When a route is needed	Route discovery
RREP	Destination → source (unicast)	654	Answer to RREQ	Route set-up
RERR	Vehicle at a broken link → sources	654	When a link breaks	Route repair
HELLO (RREP, TTL 1)	Every vehicle → neighbours	654	About every 1 s	Neighbour detection
802.11 ACK	Next hop → previous hop	–	After each unicast frame	MAC reliability

Figure 4 (Section 7.4) shows these messages for one flow: route discovery at the start, data delivery and the repair of a route after a link breaks.

7.4 Proposed Algorithm

One simulation run is the program vanet-highway.cc (Fig. 3). It takes the four factors and the seed on the command line, builds the scenario, runs 100 simulated seconds and appends one row of results to a CSV file.

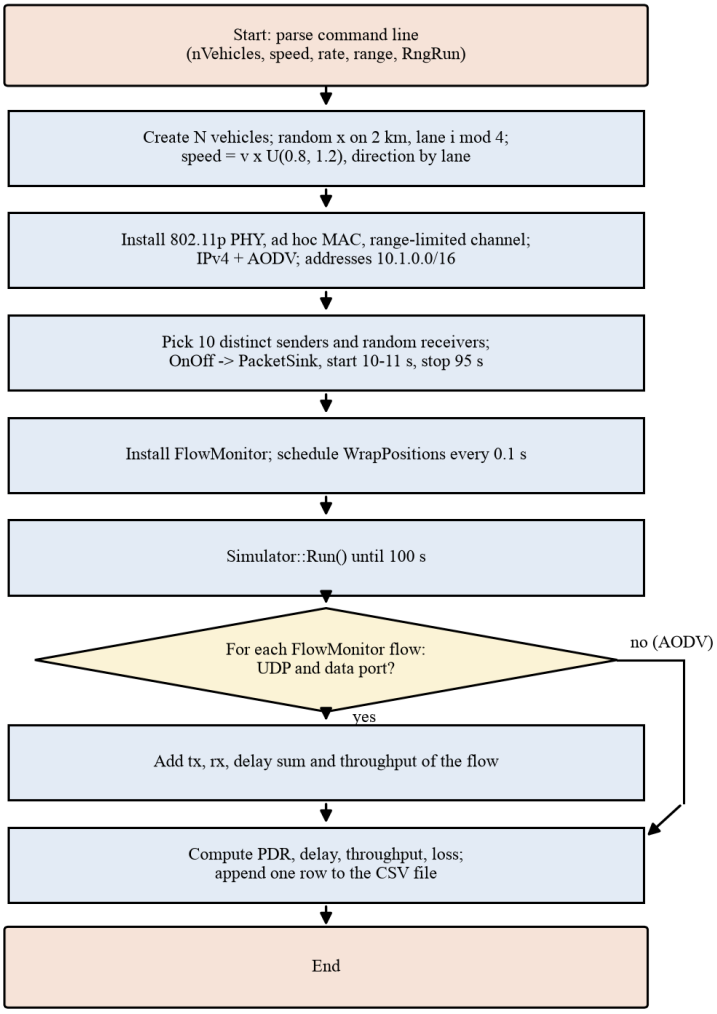


Fig. 3. Flowchart of one simulation run (vanet-highway.cc)

Algorithm 1: One simulation run (vanet-highway.cc, simplified)

```
parse N, v, R, r, RngRun from the command line
create N vehicles; for i in 0..N-1:
    lane <- i mod 4;  direction <- +1 if lane < 2 else -1
    x <- U(0, L);  speed <- v * U(0.8, 1.2)
install 802.11p PHY (6 Mbit/s), ad hoc MAC, RangePropagationLoss(r)
install IPv4 + AODV; assign addresses 10.1.0.0/16
shuffle vehicles; senders <- first 10; for each flow f:
    dst <- random vehicle != src;  port <- 9000 + f
    PacketSink on dst:port;  OnOff(src -> dst:port, rate R, 512 B)
    start <- 10 + U(0, 1);  stop <- 95
install FlowMonitor; every 0.1 s: WrapPositions()
run the simulator until t = 100 s
for each FlowMonitor flow: if UDP to a data port: add tx, rx, delay
compute PDR, delay, throughput, loss; append one row to the CSV file
```

Algorithm 2: Parameter sweeps (run_experiments.sh and plot_results.py)

```
1  for each sweep in {density, speed, load, range}:
2      for each value x of the sweep:
3          set the factor to x, keep the other three at the baseline
4          for seed in 1..SEEDS: add job (sweep, x, seed)
5  run all jobs in parallel (xargs -P JOBS); skip jobs whose CSV already
exists
6  plot_results.py: merge CSVs; mean and std per (sweep, x); draw 5 graphs
```

Algorithm 3: Finding and checking the optimal configuration

```
1  for each factor j: x_j* <- tested value with the highest mean PDR
2  c* <- (N*, v*, R*, r*);  run c* and the baseline with seeds 1..10
3  for each factor j: run c* with factor j set back to its baseline value
4  compare the measured PDR of c* with the estimate of Equation (13)
```

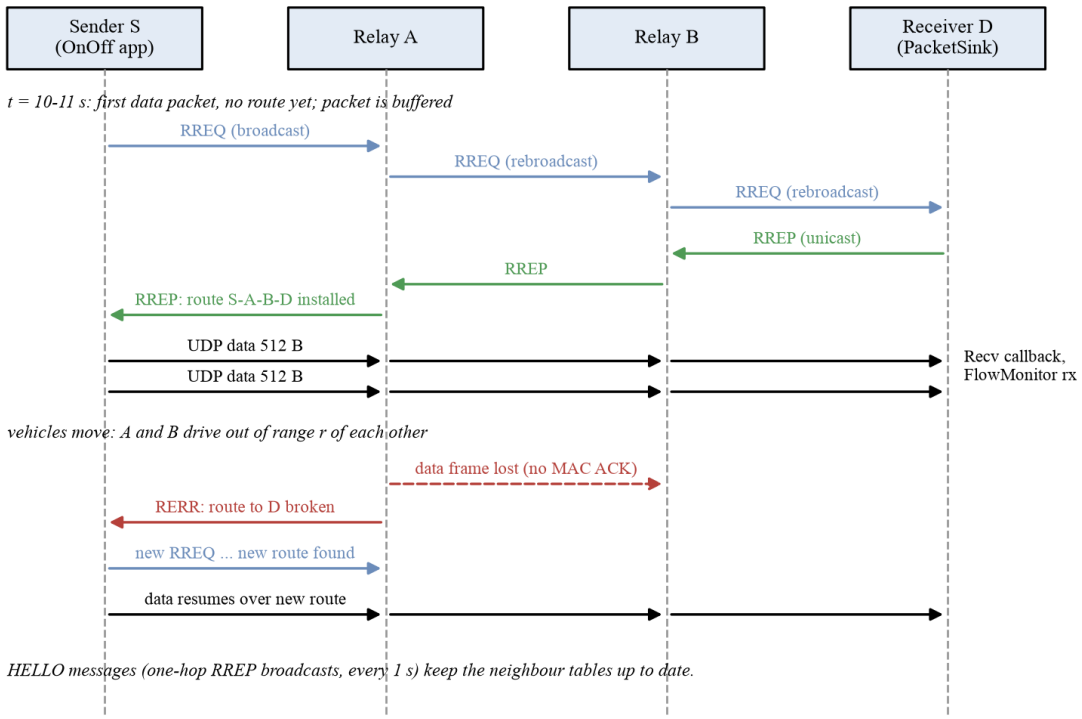


Fig. 4. Sequence diagram of route discovery, data delivery and route repair

7.5 Socket Programming Implementation

The program is a single C++ file, vanet-highway.cc (453 lines), copied into the scratch folder of ns-3 and built with ./ns3 build. The main parts are shown below.

Keeping vehicles on the road (WrapPositions)

A vehicle that leaves the road re-enters at the other end. We found that ns-3's ConstantVelocityMobilityModel::SetPosition also sets the velocity to zero, so in an early version vehicles stopped for good after their first wrap; by t = 60 s about two thirds of them

were parked. The fix restores the velocity after moving the vehicle, and all results were regenerated:

```
Vector pos = mob->GetPosition();
if (pos.x > length || pos.x < 0) {
    Vector vel = mob->GetVelocity();
    pos.x += pos.x > length ? -length : length;
    mob->SetPosition(pos);
    mob->SetVelocity(vel);   // SetPosition sets the velocity to zero
}
Simulator::Schedule(interval, &WrapPositions, vehicles, length, interval);
```

Radio: IEEE 802.11p with a hard range cut-off

```
YansWifiChannelHelper channel;
channel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
channel.AddPropagationLoss("ns3::RangePropagationLossModel",
                           "MaxRange", DoubleValue(range));
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211p);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
    "DataMode", StringValue("OfdmRate6MbpsBW10MHz"),
    "ControlMode", StringValue("OfdmRate6MbpsBW10MHz"));
mac.SetType("ns3::AdhocWifiMac");
```

Routing and addresses

```
AodvHelper aodv;
InternetStackHelper internet;
internet.SetRoutingHelper(aodv);
internet.Install(vehicles);
address.SetBase("10.1.0.0", "255.255.0.0");
```

UDP sender and receiver sockets

For each flow the receiver gets a PacketSink bound to the flow's port, and the sender an OnOffApplication that sends to the receiver's address and port at a constant rate:

```
PacketSinkHelper sink("ns3::UdpSocketFactory",
                      InetSocketAddress(Ipv4Address::GetAny(), port));
sink.Install(vehicles.Get(dst));
OnOffHelper onoff("ns3::UdpSocketFactory",
                  InetSocketAddress(interfaces.GetAddress(dst), port));
onoff.SetConstantRate(DataRate(dataRateKbps * 1000), packetSize);
ApplicationContainer srcApp = onoff.Install(vehicles.Get(src));
srcApp.Start(Seconds(appStart + startJitter->GetValue()));
srcApp.Stop(Seconds(appStop));
```

Inside ns-3 these helpers use the Socket API in the same way as a Linux program uses system calls. PacketSink calls Socket::CreateSocket() with the UDP socket factory (like *socket()*), Bind() to the port (like *bind()*) and SetRecvCallback(), and reads packets with RecvFrom() (like *recvfrom()*). OnOffApplication creates a UDP socket, calls Connect() to the receiver's

address and then Send() for each packet at intervals of 8 · S / R seconds (like *connect()* and *send()*).

Measuring only the data flows

```
for (const auto& [flowId, st] : monitor->GetFlowStats()) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(flowId);
    if (t.protocol != 17 || activeTime.count(t.destinationPort) == 0)
        continue;                       // skip AODV control traffic
    txPackets += st.txPackets;
    rxPackets += st.rxPackets;
    delaySum  += st.delaySum.GetSeconds();
    throughputSum += st.rxPackets * packetSize * 8.0
                   / activeTime[t.destinationPort] / 1000.0;
}
```

Running the program

A single run, and the full set of sweeps, are started with:

```
./ns3 run "vanet-highway --nVehicles=60 --speed=20 --dataRateKbps=100 \
           --range=150 --RngRun=1"
NS3_DIR=~/ns-allinone-3.48/ns-3.48 bash run_experiments.sh
python3 plot_results.py results
```

Two optional switches record what happens inside a run without changing the results row: --trace=FILE writes one line per event (Table 5) for the Python animation scripts, and --netanim=FILE.xml writes a NetAnim animation in which senders are orange and receivers green.

Table 5. Events written by the --trace option

Line	Fields	Meaning
F	flow, src, dst, port, start	Flow definition
P	t, node, x, y	Vehicle position (every 0.1 s)
S	t, node, port, uid	Application sends a data packet
R	t, node, port, uid, delayMs	Receiver gets a data packet
D	t, node, nextHop, port, uid	A data frame goes on air (one per hop and retry)
C	t, node, type	An AODV frame goes on air (RREQ, RREP, RERR, HELLO)

CHAPTER 8
PERFORMANCE EVALUATION

8.1 Experimental Setup

All runs were done on the same Apple Silicon laptop (macOS, 18 cores) with ns-3.48, using the settings in Table 6. NS-3 uses fixed pseudo-random number streams, so running the same command with the same --RngRun gives exactly the same result; different seeds give different start positions, speeds and flow pairs. One run takes about 25 s of CPU time, and 18 runs are done in parallel.

Table 6. Simulation parameters

Parameter	Value
Simulator	ns-3.48 (scenario written for ns-3.45), macOS
Road	Straight highway, 2000 m, 2 lanes per direction, 5 m apart, wrap-around at the ends
Mobility	Constant velocity; speed = mean × U(0.8, 1.2); half the vehicles in each direction
PHY / MAC	IEEE 802.11p (5.9 GHz, 10 MHz channel), 6 Mbit/s OFDM, 20 dBm, ad hoc MAC
Propagation	Constant-speed delay; RangePropagationLossModel, MaxRange = r
Routing	AODV (ns-3 default settings)
Traffic	10 UDP flows (OnOff → PacketSink), 512-byte payload, distinct random senders
Timing	100 s simulated; flows start at U(10, 11) s and stop at 95 s
Baseline	60 vehicles, 20 m/s, 100 kbit/s per flow, 150 m range
Seeds	5 per test point (main sweeps), 8 (second variation), 10 (optimal study)
Measurement	FlowMonitor, data flows only; one CSV row per run

8.2 Performance Metrics

The metrics in Table 7 are computed at the end of every run from FlowMonitor, using Equations (7) to (10), and averaged over the seeds of each test point. Dropped packets have no receive time, so the average delay is taken over received packets only. The flows stop at 95 s and the simulation at 100 s, so every run ends with a 5 s drain period in which packets still in queues can arrive; a packet that has not arrived by 100 s counts as lost (FlowMonitor CheckForLostPackets). Throughput is measured over the time each flow was sending, from its start time to 95 s, and counts payload bits only. The start-up period before 10 s, in which AODV learns its neighbours, carries no data and is therefore not part of any metric.

Table 7. Performance metrics

Metric	Meaning	Unit	Source
Packet delivery ratio	rx / tx × 100, over all data flows	%	FlowMonitor
End-to-end delay	Total delay of received packets / packets received	ms	FlowMonitor
Throughput	Mean over flows of payload bits received / time the flow was sending	kbit/s per flow	FlowMonitor
Packet loss rate	(tx − rx) / tx × 100	%	FlowMonitor
Mean ± std	Mean and sample standard deviation over the seeds	–	plot_results.py

8.3 Experiments and Procedure

Each sweep varies one factor and keeps the other three at the baseline (Table 8). Every test point is run with several seeds; the graphs show the mean with standard-deviation error bars. Fig. 5 shows the automated procedure. run_experiments.sh creates one job per run and runs the jobs in parallel. Each run adds one row to a CSV file, and plot_results.py merges the rows, computes the statistics and draws the graphs. Running the same command again after an interruption skips the runs that have already finished.

Table 8. Experiments

Experiment	Values	Fixed at	Seeds	Runs	Graphs
Density	20, 40, 60, 80, 100 vehicles	Baseline	5	25	Density vs PDR; density vs loss
Speed	10, 15, 20, 25 m/s (36–90 km/h)	Baseline	5	20	Speed vs delay
Load	50, 100, 250, 500, 750 kbit/s per flow	Baseline	5	25	Load vs throughput
Range	50, 100, 150, 200, 250 m	Baseline	5	25	Range vs PDR
Second variation	Density 30–130, speed 5–35, load 25–1000, range 75–325	70 veh., 15 m/s, 150 kbit/s, 175 m	8	200	Same four graphs
Optimal study	Optimal, baseline and 4 leave-one-out configurations	See Section 9.4	10	60	PDR, delay, loss bars

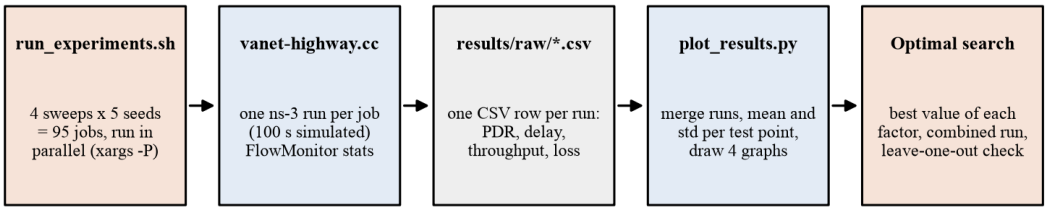


Fig. 5. Steps from the job list to the graphs and the optimal configuration

8.4 Fairness and Validity of the Comparison

Within a sweep, every test point uses seeds 1 to 5, so the same random streams set the start positions, speeds and flow pairs and only the varied factor changes. The baseline appears in all four sweeps and gives 36.98% PDR each time. It is the same configuration with the same seeds, so this shows only that the runs are repeatable. The second variation changes the baseline and the tested values and uses eight seeds, which tests whether the trends depend on our first choices. The optimal configuration and the baseline were run with the same ten seeds, so they are compared on the same random streams. Over those ten seeds the baseline averages 36.0%. The 36.98% figure is the mean of seeds 1 to 5 only, and seeds 6 to 10 average 35.1%.

Two limits apply to all results. Results vary a lot from run to run: with 5 seeds the standard deviation of PDR is about 3 to 15 points, so neighbouring test points can overlap. The radio model also has a hard range and no fading, so the absolute PDR values apply only to this model and we compare trends.

CHAPTER 9
OUTPUT AND RESULTS

9.1 Comparative Analysis

Table 9 gives the mean and standard deviation for each test point in the four main sweeps, using 5 seeds per point. The baseline (60 vehicles, 20 m/s, 100 kbit/s, 150 m) appears once in each sweep.

Table 9. Results of the four parameter sweeps (mean ± standard deviation, 5 seeds)

Sweep	Value	PDR (%)	Delay (ms)	Throughput (kbit/s/flow)	Loss (%)
Density	20 veh.	21.3 ± 8.7	229.5 ± 64.2	21.3 ± 8.7	78.7
Density	40 veh.	28.8 ± 5.6	153.4 ± 54.2	28.8 ± 5.6	71.2
Density	60 veh.	37.0 ± 11.3	67.4 ± 29.7	37.0 ± 11.3	63.0
Density	80 veh.	37.5 ± 6.2	32.1 ± 19.8	37.5 ± 6.2	62.5
Density	100 veh.	29.1 ± 6.7	37.7 ± 10.6	29.1 ± 6.7	70.9
Speed	10 m/s	46.1 ± 14.7	58.0 ± 31.7	46.1 ± 14.7	53.9
Speed	15 m/s	41.5 ± 12.5	76.4 ± 16.0	41.5 ± 12.5	58.5
Speed	20 m/s	37.0 ± 11.3	67.4 ± 29.7	37.0 ± 11.3	63.0
Speed	25 m/s	32.3 ± 9.5	96.3 ± 23.5	32.3 ± 9.5	67.7
Load	50 kbit/s	39.2 ± 12.3	151.1 ± 71.2	19.6 ± 6.2	60.8
Load	100 kbit/s	37.0 ± 11.3	67.4 ± 29.7	37.0 ± 11.3	63.0
Load	250 kbit/s	29.7 ± 13.4	59.8 ± 22.7	74.2 ± 33.5	70.3
Load	500 kbit/s	24.7 ± 12.3	61.0 ± 16.9	123.4 ± 61.5	75.3
Load	750 kbit/s	19.2 ± 9.4	77.2 ± 37.0	143.9 ± 70.3	80.8
Range	50 m	5.8 ± 2.9	360.5 ± 212.9	5.8 ± 2.9	94.2
Range	100 m	18.7 ± 6.8	117.9 ± 32.4	18.7 ± 6.8	81.3
Range	150 m	37.0 ± 11.3	67.4 ± 29.7	37.0 ± 11.3	63.0
Range	200 m	45.9 ± 15.1	41.5 ± 33.4	45.9 ± 15.1	54.1
Range	250 m	53.1 ± 11.6	34.1 ± 29.7	53.0 ± 11.6	46.9

Vehicle density

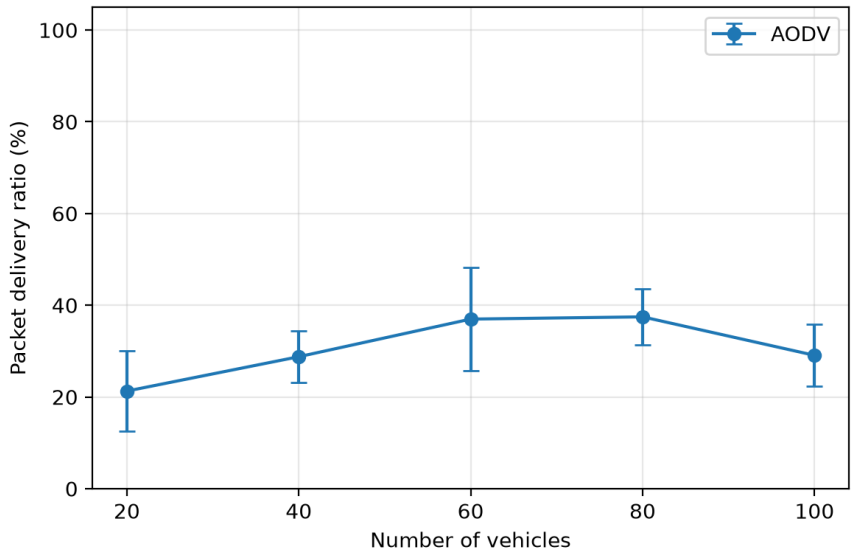


Fig. 6. Vehicle density versus packet delivery ratio (AODV; 20 m/s, 100 kbit/s, 150 m; 5 seeds)

PDR rises from 21.3% with 20 vehicles to 37.0% with 60 and 37.5% with 80, then falls to 29.1% with 100 vehicles (Fig. 6). With 20 vehicles a vehicle has only about 3 neighbours (Equation 3), the road often splits into groups that cannot reach each other and packets wait for routes that never form; the delay is 229.5 ms. More vehicles fill the gaps and the delay falls to 32.1 ms at 80. With 100 vehicles each RREQ flood reaches about 15 neighbours, and route discovery and channel contention start to cost more than the extra connectivity gives [11].

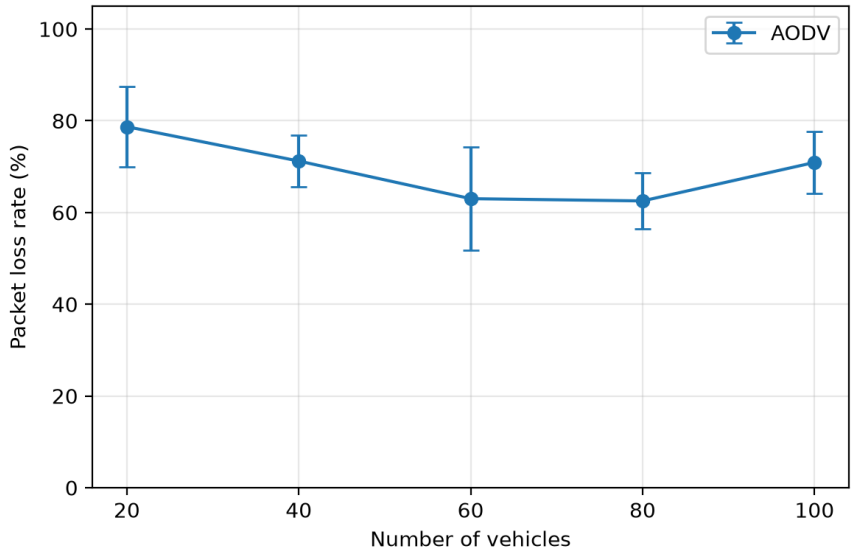


Fig. 7. Vehicle density versus packet loss rate (AODV; 20 m/s, 100 kbit/s, 150 m; 5 seeds)

Packet loss is the mirror image of PDR (Fig. 7): it falls from 78.7% with 20 vehicles to 62.5% with 80 and rises again to 70.9% with 100. Even at its lowest, more than 60% of the packets are lost at the baseline speed, load and range, which shows how hard multi-hop delivery along a 2 km road is.

Vehicle speed

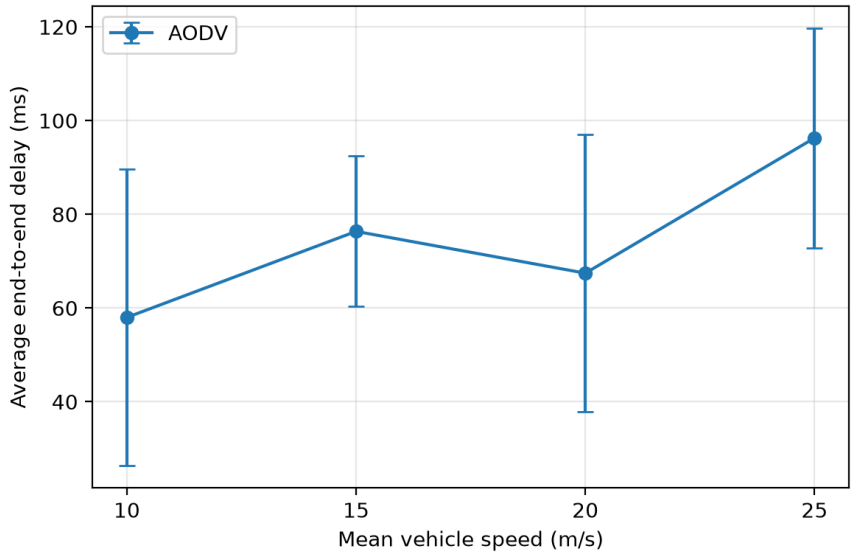


Fig. 8. Average end-to-end delay against mean vehicle speed (AODV; 60 vehicles, 100 kbit/s, 150 m; 5 seeds)

Delay rises from 58.0 ms at 10 m/s to 96.3 ms at 25 m/s (Fig. 8). The dip at 20 m/s lies inside the error bars. PDR falls steadily from 46.1% to 32.3%. A likely cause of both is that links between vehicles break sooner (Equation 5), so AODV spends more time rediscovering routes while packets are buffered or dropped. Route-error counts at each speed would be needed to confirm this.

Traffic load

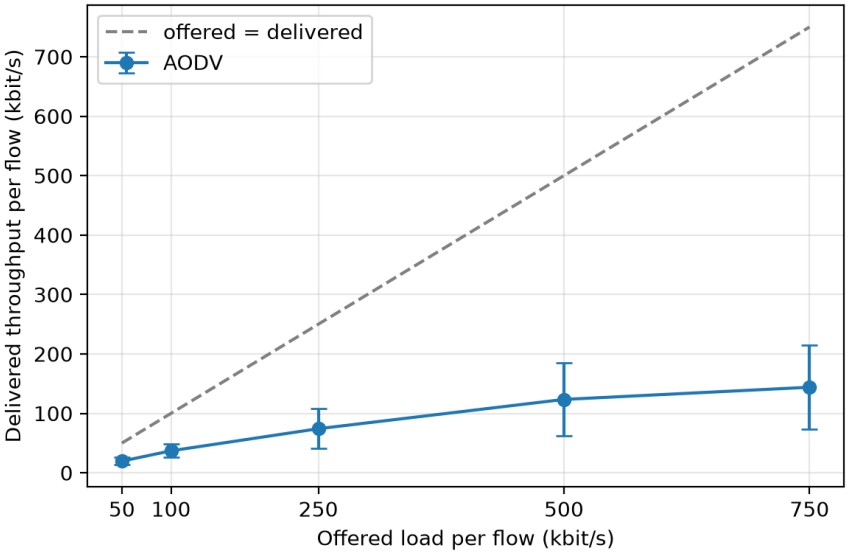


Fig. 9. Offered load versus delivered throughput per flow (AODV; 60 vehicles, 20 m/s, 150 m; 5 seeds)

Delivered throughput grows from 19.6 to 143.9 kbit/s per flow as the offered load grows from 50 to 750 kbit/s, but it falls further and further below the offered rate (dashed line, Fig. 9): about 39% of the offered rate arrives at 50 kbit/s and only 19.2% at 750 kbit/s. The shared 6 Mbit/s channel saturates (Equation 6), so more packets collide or are dropped from full queues. The delay at 50 kbit/s (151.1 ± 71.2 ms) is higher than at 100 kbit/s; we have not found the reason, and the error bar is large.

Communication range

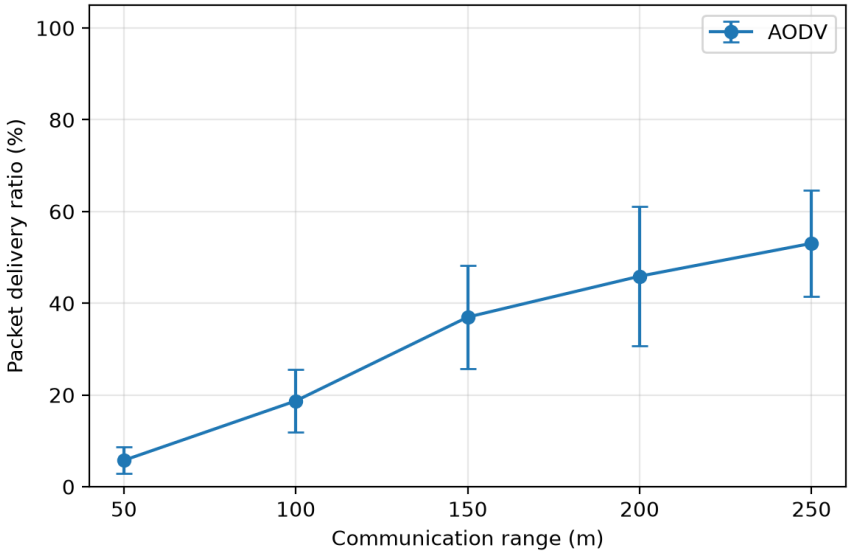


Fig. 10. Communication range versus packet delivery ratio (AODV; 60 vehicles, 20 m/s, 100 kbit/s; 5 seeds)

Range has the strongest effect. PDR rises from 5.8% at 50 m to 53.1% at 250 m, and the delay falls from 360.5 ms to 34.1 ms (Fig. 10). At 50 m a vehicle has about 3 neighbours and a route would need about 13 hops (Equation 4), so the road splits into isolated groups and most packets never find a route. A longer range gives more neighbours and shorter routes.

Comparison of the factors

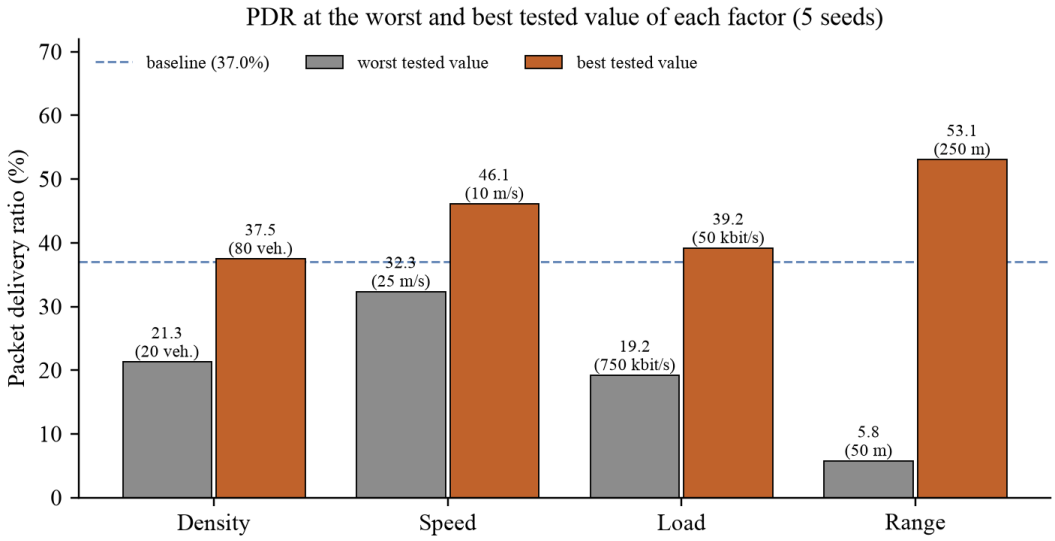


Fig. 11. PDR at the worst and best tested value of each factor

Table 10. Effect of each factor on PDR

Factor	Tested	Worst PDR (%)	Best PDR (%)	Span (points)	Best vs baseline
Range	50–250 m	5.8 (50 m)	53.1 (250 m)	47.3	+16.1
Load	50–750 kbit/s	19.2 (750 kbit/s)	39.2 (50 kbit/s)	20.0	+2.2
Density	20–100 veh.	21.3 (20 veh.)	37.5 (80 veh.)	16.2	+0.5
Speed	10–25 m/s	32.3 (25 m/s)	46.1 (10 m/s)	13.8	+9.1

Range changes PDR the most, both over the tested values (47 points) and as an improvement over the baseline (+16 points). Speed is the second-best way to improve on the baseline (+9 points). High load and low density hurt a lot, but going below the baseline load or above the baseline density helps only a little (Fig. 11, Table 10).

9.2 Validation of the Approach

We checked the results in seven ways (Table 11).

Table 11. Validation checks

Check	What was done	Result
Repeatability	Baseline seeds 1–5 run again for this report	Identical to the last digit (seed 1: 20599 sent, 7416 received)
Shared baseline	Baseline point compared across the four sweeps	Same 36.98% PDR in all four
Bug fix	Vehicles stopped after wrapping (velocity reset to zero)	Velocity restored; every result regenerated
Metric consistency	Throughput compared with PDR × offered rate	Load 250: 29.7% × 250 = 74.2 ≈ 74.2 kbit/s
PDR and loss	PDR + loss compared for every run (95 + 200 + 60 runs)	Sum is 100% in every run (to within 0.001 points)
Control traffic	Only UDP flows to the data ports counted	AODV flows excluded from all metrics
Second variation	New baseline, wider ranges, 8 seeds (200 runs)	Same trends (Fig. 12, Table 12)

Second parameter variation

To check that the trends do not depend on our choice of baseline and values, we ran a second variation with a baseline of 70 vehicles, 15 m/s, 150 kbit/s and 175 m, wider ranges of values and eight seeds per point (Fig. 12, Table 12).

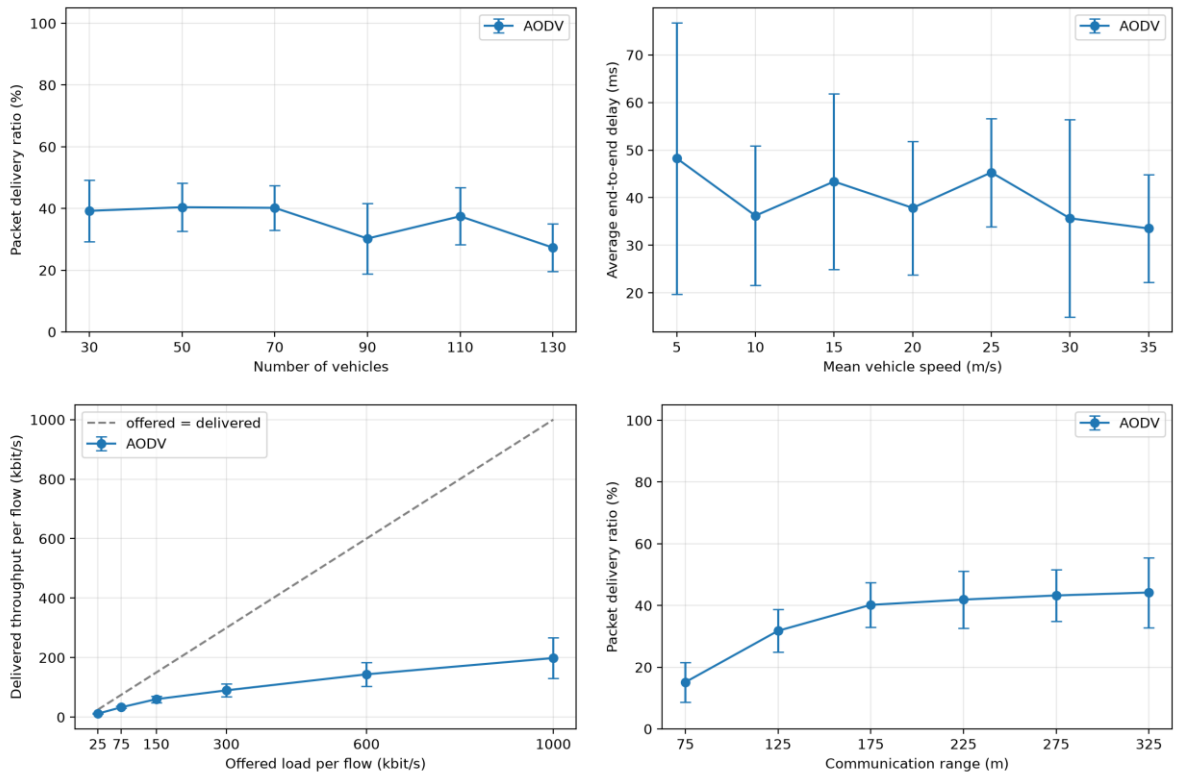


Fig. 12. Second variation (8 seeds): density vs PDR, speed vs delay, load vs throughput and range vs PDR

Table 12. PDR (%) in the second variation, in the order of the tested values

Sweep	Values	PDR (%)
Density	30–130 veh.	39.2 / 40.4 / 40.2 / 30.3 / 37.5 / 27.3
Speed	5–35 m/s	44.8 / 43.7 / 40.2 / 36.2 / 33.2 / 32.4 / 31.0
Load	25–1000 kbit/s	46.4 / 44.1 / 40.2 / 30.0 / 24.0 / 19.9
Range	75–325 m	15.2 / 31.8 / 40.2 / 41.9 / 43.3 / 44.2

The trends are the same. PDR falls steadily with speed (44.8% at 5 m/s to 31.0% at 35 m/s) and with load (46.4% to 19.9%). Density is flat up to 70 vehicles and then falls. Range helps most at short distances (15.2% at 75 m to 40.2% at 175 m). Above about 225 m PDR levels off (41.9%, 43.3%, 44.2%) and the delay rises again (45.2 to 71.9 ms). Once the road is connected, extra range probably adds contention, although we did not measure retransmissions or channel busy time.

9.3 Visualisation of the Sweeps

We used make_animations.sh to run seed 1 of every test point with tracing on. animate_sweep.py then draws all the values of a sweep on one clock, next to a live bar of the metric that the sweep's graph plots. Figures 13 and 14 show the state at t = 30 s. Orange

squares are senders, blue arcs are data hops on air, light blue dots are route requests, red crosses are route errors and green circles are delivered packets.

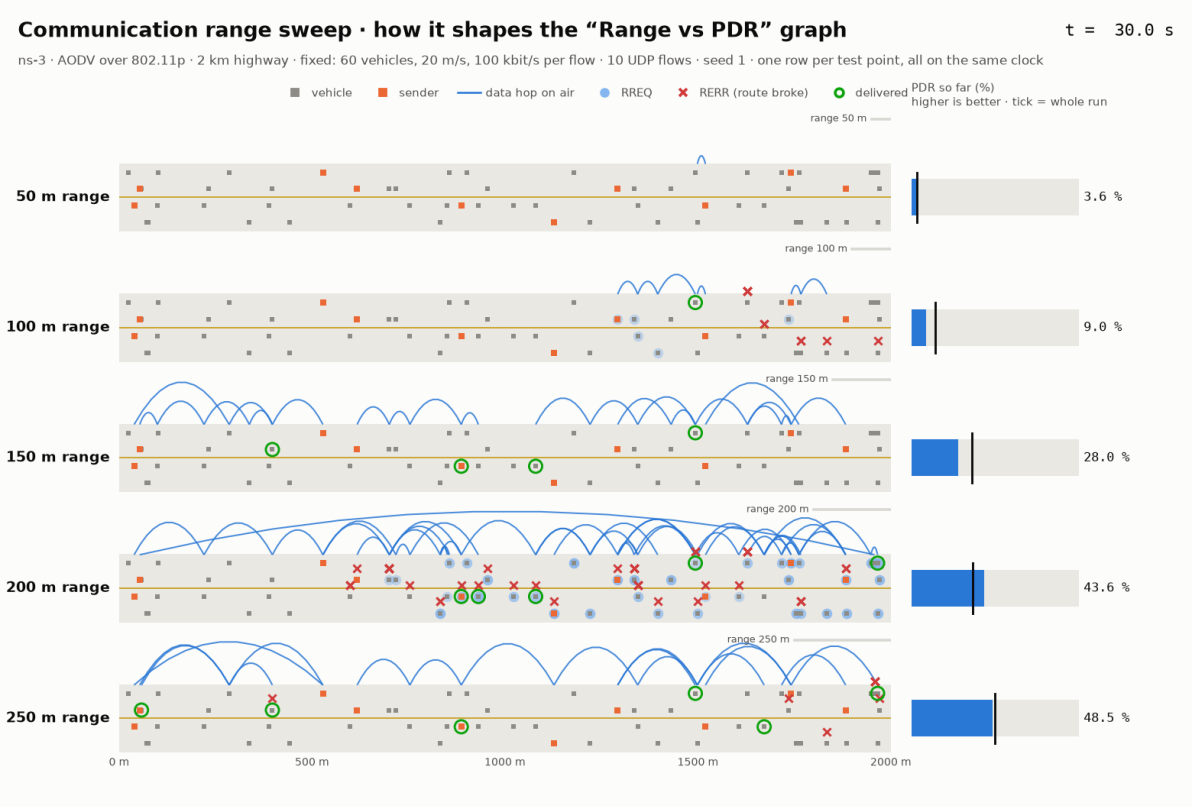


Fig. 13. Range sweep at t = 30 s (seed 1): hops get longer and deliveries increase as the range grows

At 50 m (top row) hardly any data hops are on air, because senders cannot find routes. As the range grows, the arcs get longer and more packets are delivered. At 200 m many route errors (red) appear as links break and are repaired.

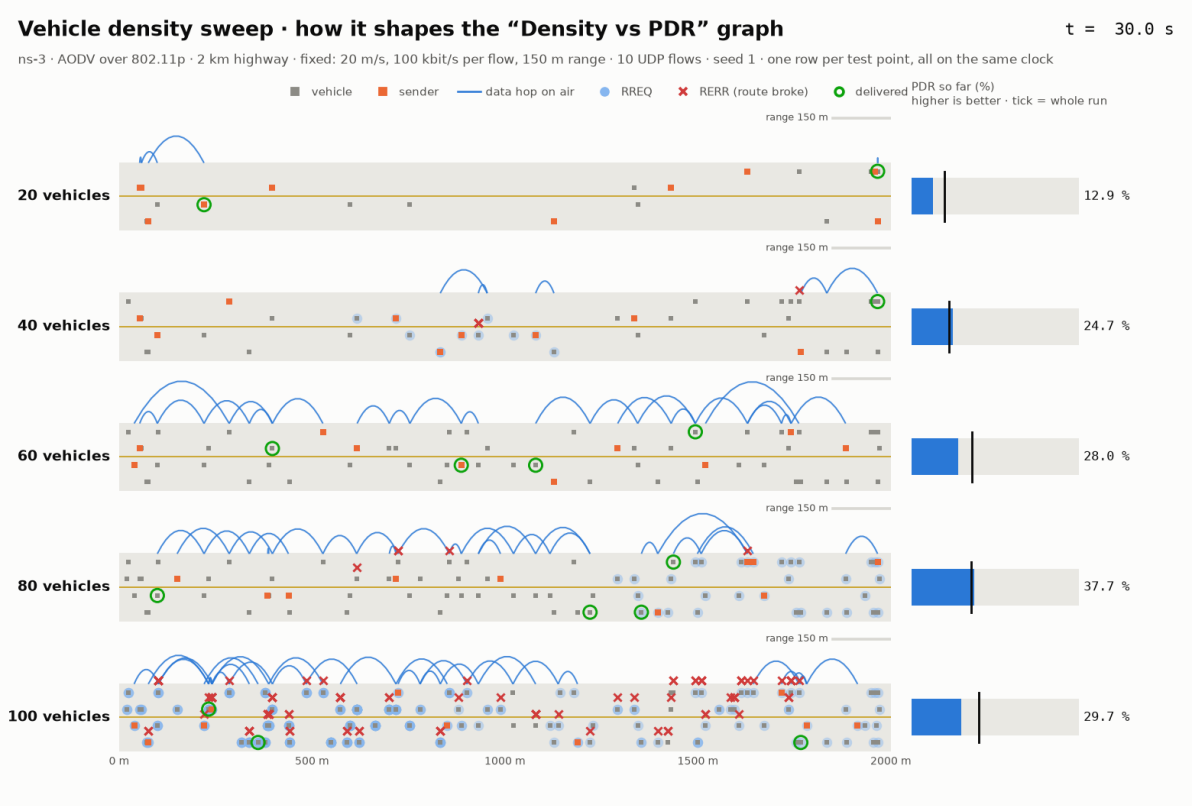


Fig. 14. Density sweep at t = 30 s (seed 1)

With 20 vehicles the road has long empty gaps and the senders are isolated. With more vehicles the gaps close and more packets are delivered. All rows use the same seed, so a single run can differ from the 5-seed mean: with seed 1, 100 vehicles reach 40% PDR, although the 5-seed mean drops to 29% there.

9.4 Final Optimal Solution

Table 13 lists the best tested value of each factor, using the 5-seed means of Table 9.

Table 13. Best tested value of each factor (5 seeds)

Factor	Best value	PDR (%)	Delay (ms)	Loss (%)	Why
Density	80 vehicles	37.5	32.1	62.5	Sweet spot: 20 is too sparse, 100 has too much contention
Load	50 kbit/s (lowest)	39.2	151.1	60.8	Less traffic means less contention; throughput is lower
Range	250 m (highest)	53.1	34.1	46.9	More neighbours, fewer hops, no isolated groups
Speed	10 m/s (lowest)	46.1	58.0	53.9	Links last longer, fewer route breaks

Combining them gives the candidate optimal configuration c* = (80 vehicles, 10 m/s, 50 kbit/s, 250 m). If the factors did not interact, Equation (13) would predict a PDR of 37.0% ×

1.013 × 1.059 × 1.435 × 1.247 ≈ 71.0%. Instead of relying on this estimate, we ran c* and the baseline in the simulator with the same ten seeds (Table 14, Fig. 15).

Table 14. Baseline and optimal configuration (mean ± standard deviation, 10 seeds)

Measure	Baseline	Optimal	Change
Vehicles / speed	60 / 20 m/s	80 / 10 m/s	–
Load / range	100 kbit/s / 150 m	50 kbit/s / 250 m	–
Packet delivery ratio	36.0 ± 8.3%	59.1 ± 3.6%	+23.1 points (× 1.64)
Average delay	69.2 ± 25.1 ms	20.7 ± 4.9 ms	−70%
Packet loss rate	64.0%	40.9%	−23.1 points
Throughput per flow	36.0 kbit/s (of 100)	29.6 kbit/s (of 50)	Lower: half the offered load
Estimate, Equation (13)	–	71.0% PDR	Measured is 11.8 points lower

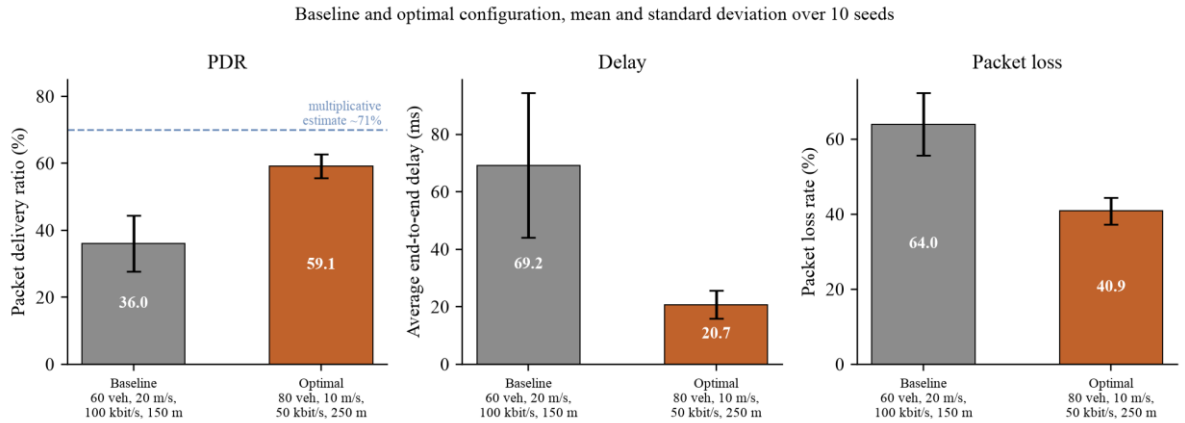


Fig. 15. PDR, delay and packet loss of the baseline and the optimal configuration (10 seeds)

The optimal configuration raises PDR from 36.0% to 59.1% and cuts the average delay by about 70%, from 69.2 ms to 20.7 ms, well inside the 100 ms that safety applications need. It is also far more predictable: the standard deviation of PDR falls from 8.3 to 3.6 points, and it beats the baseline on every one of the ten seeds (Fig. 16). However, the measured PDR is about 12 points below the 71% estimate, and only 6 points above the best single change (250 m range alone, 53.1%). The gains of the factors do not multiply.

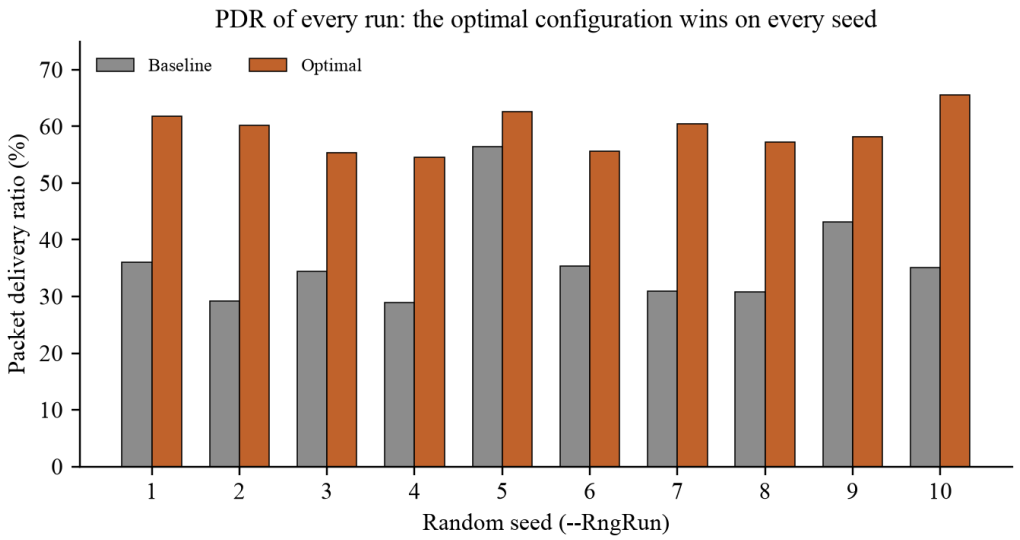


Fig. 16. PDR of every run of the baseline and the optimal configuration

Leave-one-out check

To see how much each factor contributes once the others are already at their best, we ran c* four more times, each time putting one factor back to its baseline value (Table 15, Fig. 17).

Table 15. Leave-one-out check (10 seeds)

Factor put back	N, v, R, r	PDR (%)	Delay (ms)	Throughput (kbit/s/flow)	PDR vs optimal
None (optimal)	80, 10, 50, 250	59.1 ± 3.6	20.7 ± 4.9	29.6	–
Density 80 → 60	60, 10, 50, 250	63.3 ± 8.8	20.5 ± 10.7	31.6	+4.2
Speed 10 → 20 m/s	80, 20, 50, 250	52.9 ± 6.6	21.8 ± 6.1	26.5	−6.2
Load 50 → 100 kbit/s	80, 10, 100, 250	51.4 ± 5.8	36.3 ± 17.1	51.4	−7.8
Range 250 → 150 m	80, 10, 50, 150	51.8 ± 5.5	37.2 ± 17.1	25.9	−7.4
All four (baseline)	60, 20, 100, 150	36.0 ± 8.3	69.2 ± 25.1	36.0	−23.1

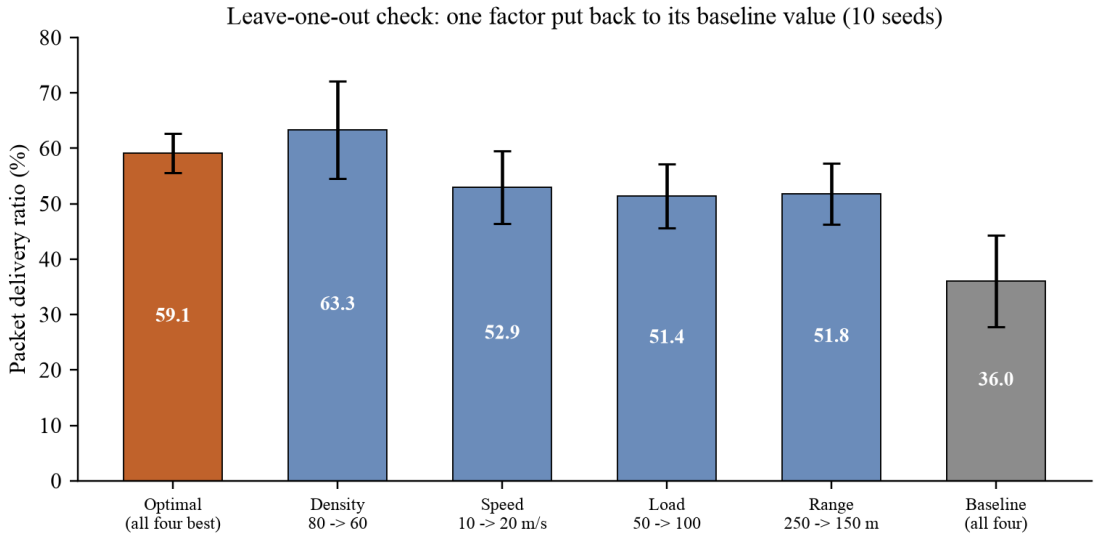


Fig. 17. Leave-one-out check: PDR when one factor is put back to its baseline value

Putting speed, load or range back each costs 6 to 8 PDR points and raises the delay (to about 36.3 ms for load and 37.2 ms for range), so these three factors all still help in the combination. Density is different: with 60 instead of 80 vehicles the PDR is 63.3 ± 8.8%, slightly higher than 59.1 ± 3.6% but well within one standard deviation. With a 250 m range each vehicle already has about 15–20 neighbours, so more vehicles may add contention without improving connectivity much. That would move the best density down as the range grows, and this interaction is one likely reason why the single-factor gains do not multiply.

Recommended configuration

Final optimal solution. A communication range of 250 m, a mean speed of 10 m/s, an offered load of 50 kbit/s per flow and 60–80 vehicles per 2 km of four-lane highway. The choice between 60 and 80 vehicles is a trade-off. Sixty vehicles have the higher mean PDR (63.3% against 59.1%) and do better on six of the ten seeds, but the difference is within the error bars. Eighty vehicles are more consistent (standard deviation 3.6 against 8.8 points, lowest seed 54.6% against 51.3%), so we keep 80 as the final value, although ten seeds give only a rough estimate of that spread. Over ten seeds, 80 vehicles deliver 59.1% of the packets with an average delay of 20.7 ms and a loss rate of 40.9%.

Only two of the four factors are design choices. The load depends on how often applications send. On real hardware the range would depend on the transmit power and the receiver sensitivity, although our model sets it directly with a hard cut-off. Speed and density describe the traffic, so the result says which conditions suit the network best, for example dense, slower traffic on a speed-limited stretch of highway. If throughput matters more than reliability, a higher load is better: at 100 kbit/s per flow the network delivers 51.4 kbit/s per flow instead of 29.6, at the cost of 8 PDR points.

9.5 Observations

- Over the values we tested, range has the largest effect on PDR: a span of 47 points for a fivefold change in range, against 14 points for a 2.5-fold change in speed. This ranking depends on the values tested. A short range splits the highway into isolated groups, and a longer range gives more neighbours and fewer hops. Above about 225 m the gain levels off and the delay rises again.
- Faster vehicles lose more packets and their packets take longer to arrive. The likely reason is that links last a shorter time, so AODV has to find new routes more often.

- Channel saturation explains only part of the loss. At 50 kbit/s per flow, the lightest load tested, 60.8% of the packets are still lost, which points to missing or broken routes. As load grows and the shared channel fills, PDR falls further, from 37.0% at the baseline to 19.2% at 750 kbit/s per flow.
- Density has a best region around 60–80 vehicles. Too few vehicles leave gaps in the road. With too many we expect more route-discovery traffic and contention, although we did not count AODV messages. At a 250 m range 60 vehicles do as well as 80, so the best density may fall as the range grows.
- A lower loss rate does not always come with a lower delay. At 50 kbit/s per flow, PDR is the highest in the load sweep and so is the delay (151.1 ms).
- Run-to-run variation is large, so several seeds are essential. A single seed can even show the opposite of the trend: at 100 vehicles seed 1 gives 40.3% PDR against a five-seed mean of 29.1%, and it is above seed 1 at 60 and 80 vehicles (36.0% and 35.8%).
- The combination of the best values works (PDR 36.0% to 59.1%), but the gains do not add up: the measured PDR is about 12 points below the multiplicative estimate.

CHAPTER 10
CONCLUSION

Main findings. In our highway VANET with AODV over IEEE 802.11p, communication range had the strongest effect on delivery: PDR rose from 5.8% at 50 m to 53.1% at 250 m and the delay fell from 360.5 ms to 34.1 ms. Higher speed lowered PDR (46.1% to 32.3%) and raised the delay (58.0 to 96.3 ms). A higher load raised the delivered throughput (19.6 to 143.9 kbit/s per flow) but lowered the share delivered (39.2% to 19.2%). Density helped up to 60–80 vehicles and then hurt. A second variation with a different baseline and eight seeds showed the same trends for speed, load and range, and the gain from range levelled off above about 225 m. Density behaved a little differently: PDR peaks at 60–80 vehicles in the main sweep, while in the second variation it stays flat up to 70 vehicles and then falls.

Optimal solution. Combining the best tested value of each factor (80 vehicles, 10 m/s, 50 kbit/s per flow, 250 m) and running it over ten seeds raised PDR from 36.0% to 59.1%, cut the delay from 69.2 ms to 20.7 ms and reduced the standard deviation of PDR from 8.3 to 3.6 points. It is 12 points below the 71% that multiplying the single-factor gains predicts. The leave-one-out runs show why: speed, load and range each still add 6–8 points, but density no longer helps once the range is long, and 60 vehicles do as well as 80.

Contribution. We built a reproducible 802.11p and AODV highway scenario for current ns-3 without the removed wave module, measured the four factors with several seeds and error bars, found and fixed a mobility bug that had stopped most vehicles, and measured, rather than estimated, the combined optimal configuration and the contribution of each factor to it. This answers the research gap chosen in Chapter 5.

Limitations. (1) The radio reaches exactly r metres; there is no fading, shadowing or obstacles, so the absolute PDR values are specific to this model. (2) Vehicles drive at constant velocity on a straight road; there is no car-following, lane changing or junctions. (3) Traffic is unicast constant-rate UDP between random pairs, whereas real safety applications send periodic broadcast beacons, so our results apply to safety traffic only indirectly. The 20.7 ms delay of the optimal configuration meets the 100 ms target, for example, but only 59.1% of its packets arrive. (4) We tested only AODV, with default settings. (5) We used the ad hoc MAC in place of the 802.11p OCB mode. (6) We searched for the optimum one factor at a time over a few tested values, so a full grid could find a better combination. (7) We could not explain the delay peak at 50 kbit/s. (8) The error bars remain wide even with 5–10 seeds.

(9) These are network metrics. They show whether packets arrive reliably and on time, and they cannot show whether real crashes or congestion go down.

Future work. (a) Replace the hard range with a realistic propagation model such as Nakagami fading (limitation 1). (b) Drive the vehicles with road traffic from SUMO [19] through ns-3's trace support (limitation 2). Veins [18] couples SUMO with OMNeT++, so using it would mean leaving ns-3. (c) Compare AODV with other routing protocols such as OLSR, DSDV and GPSR (limitation 4), and test broadcast safety beacons (limitation 3). (d) Search the four factors jointly on a grid around the optimum, with more seeds (limitations 6 and 8). (e) Add roadside units, which our scenario does not have, and then test on real 802.11p hardware.